



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования
«Дальневосточный федеральный университет»
(ДВФУ)

ИНСТИТУТ МАТЕМАТИКИ И КОМПЬЮТЕРНЫХ
ТЕХНОЛОГИЙ

Кафедра математического и компьютерного моделирования

Кулахсзян Сергей Грайрович, Талмазан Анастасия Александровна
ПРОЕКТ ПО WEB-ПРОГРАММИРОВАНИЮ

КУРСОВОЙ ПРОЕКТ

Направление подготовки 02.03.01сст Сквозные цифровые технологии,
бакалавриатская программа «Математика и компьютерные науки»
Очной (заочной) формы обучения

Студенты группы Б9123-02.03.01сст

_____ С. Г. Кулахсзян

_____ А. А. Талмазан

Руководитель проекта

_____ И. С. Дроздова

Владивосток
2025

Содержание

1	Введение	2
2	Frontend	3
2.1	Математическое приложение	3
2.1.1	Конвертирование валюты	3
2.1.2	Расчёт возраста	3
2.1.3	Анимация загрузки	4
2.2	Хранение пользовательских данных	5
2.2.1	Определение	6
2.2.2	Структура JWT	6
3	Backend	7
3.1	Структура базы данных	7
3.2	Подключенные инструменты	7
3.2.1	Simple JWT	7
3.2.2	Simple Mail Transfer Protocol	9
3.2.3	Celery	10
3.2.4	Stripe API	10
3.3	Роуты для обращения к API	10
3.3.1	currency	11
3.3.2	feedback	12
3.3.3	games	16
3.3.4	library	27
3.3.5	payments	29
3.3.6	users	30

1 Введение

Данный проект представляет собой веб-приложение "Магазин видеоигр" разработанное с использованием серверного фреймворка Django на языке программирования Python и клиентской библиотеки React на языке программирования JavaScript.

Основная цель проекта — создание удобного онлайн-магазина, где пользователи могут просматривать, выбирать и приобретать видеоигры.

Основные задачи проекта:

- Реализовать серверную часть на Django для обработки запросов, работы с базой данных и предоставления API;
- Разработать клиентскую часть на React для создания удобного пользовательского интерфейса;
- Настроить взаимодействие фронтенда и бэкенда через REST API;
- Реализовать авторизацию, аутентификацию пользователя, а также возможность выхода из аккаунта и его удаление;
- Получить обширную информацию о видеоиграх, странах и основных валютах, соответствующие стране, и курсе валют для формирования базы данных и дальнейшего применения этой информации;
- Добавить платёжную систему для совершения сделки купли-продажи пользователем продукта;
- Добавить возможность создания заявлений пользователем;

2 Frontend

2.1 Математическое приложение

2.1.1 Конвертирование валюты

В базе данных цены на игры хранятся в центах, это позволяет избегать хранения информации в дробных числах, из-за чего расчёты будут более точными.

У каждого пользователя цены на игры будут отображаться в той валюте, которая будет основной в стране, которую он выбрал при регистрации. Для корректного отображения необходимо конвертировать центы в корректную валюту. Это делается по формуле:

$$S_{\text{currency}} = S_{\text{USD}} \times K_{\text{currency}},$$

$$S_{\text{USD}} = \frac{S_{\text{cents}}}{100},$$

где S_{cents} – значение цены в центах, S_{USD} – значение в долларах, K_{currency} – соотношение курса избранной валюты к доллару, S_{currency} – значение цены в избранной валюте.

2.1.2 Расчёт возраста

У всех игр есть возрастной рейтинг, благодаря которому определяется минимальный возраст, допустимый для приобретения продукта, согласно законодательству. Для того, чтобы знать при каких случаях стоит предотвращать доступ к пользователю к определённому товару, необходимо рассчитать возраст пользователя.

Возраст рассчитывается в полных годах и зависит от текущей даты. Для этого используется следующая формула:

$$A = Y_{\text{today}} - Y_{\text{birth}},$$

где Y_{today} – текущий год, Y_{birth} – год рождения пользователя, A – возраст пользователя в полных годах.

Однако необходимо учитывать, что если день и месяц текущей даты ещё не достигли дня и месяца рождения пользователя в текущем году, возраст должен быть уменьшен на единицу. Условие корректировки записывается следующим образом:

$$A = A - 1, \quad \text{если } (M_{\text{today}} < M_{\text{birth}}) \text{ или } (M_{\text{today}} = M_{\text{birth}} \text{ и } D_{\text{today}} < D_{\text{birth}}),$$

где M_{today} – текущий месяц, M_{birth} – месяц рождения пользователя, D_{today} – текущее число, D_{birth} – число рождения пользователя.

2.1.3 Анимация загрузки

Часто на сайте изображения долго грузятся, дабы не было никакого пустого пространства на сайте, было принято решение добавить анимацию загрузки, которая представляет из себя движение 8 кругов по окружности друг за другом.

Создан компонент, представляющий собой анимацию, в которой 8 кругов вращаются вокруг центра по окружности, начиная с разных позиций и плавно появляется. Рассмотрим работу этого компонента более детально.

Для создания 8 элементов (кругов) используется следующий цикл:

$$Array.from(\text{length} : 8).map((_, \text{index}) \Rightarrow \dots)$$

Этот массив из 8 элементов проходит цикл, и для каждого индекса генерируется элемент типа `div` с уникальными стилями. Основная цель заключается в том, чтобы отобразить 8 кругов, которые будут расположены равномерно по окружности.

Каждому кругу присваивается угловая трансформация, которая зависит от его индекса в массиве. Стил для каждого круга выглядит так:

$$\text{transform} : \text{rotate} \left(\frac{360}{8} \times \text{index} \right) \text{deg}, \text{translateX}(150\%), \text{rotate} \left(-\frac{360}{8} \times \text{index} \right) \text{deg}$$

Где:

- $\text{rotate} \left(\frac{360}{8} \times \text{index} \right)$ — этот угол позволяет расположить каждый круг на равном расстоянии друг от друга по окружности (360 градусов делятся на 8 частей).
- $\text{translateX}(150\%)$ — этот параметр сдвигает круг от его исходной позиции на 150% от его радиуса, таким образом круги будут располагаться по внешней окружности.
- $\text{rotate} \left(-\frac{360}{8} \times \text{index} \right)$ — поворачивает каждый круг на противоположный угол, чтобы сохранить правильную ориентацию элементов.

Таким образом, каждый круг будет располагаться на окружности, и их движение будет синхронизировано.

Для анимации вращения каждого круга используется следующий CSS-код:

```

1 @keyframes orbit {
2   0% {
3     transform: rotate(0deg) translateX(150%)
4       rotate(0deg);
5   }
6   100% {
7     transform: rotate(360deg) translateX(150%)
8       rotate(-360deg);
9   }
10 }

```

В начале анимации (0%) каждый круг находится в своей начальной позиции. В конце анимации (100%) круг проходит полный оборот (360 градусов) вокруг центра и возвращается на своё начальное место.

Для плавного появления каждого круга используется анимация изменения прозрачности:

```

1 @keyframes fade-in {
2   0% {
3     opacity: 0;
4   }
5   100% {
6     opacity: 1;
7   }
8 }

```

Каждому кругу присваивается задержка анимации, которая зависит от его индекса:

```

1 animation: orbit 1.5s linear infinite, fade-in 0.19s
  {index} * 0.19s forwards;

```

Анимация вращения (`orbit`) длится 1.5 секунды и выполняется бесконечно. Анимация появления (`fade-in`) имеет задержку, которая зависит от индекса элемента, создавая эффект последовательного появления элементов.

2.2 Хранение пользовательских данных

После регистрации или авторизации для корректной работы сервиса необходимо в каком-то виде хранить информацию о пользователе. Для этого я применяю токены.

2.2.1 Определение

JSON Web Token (JWT) представляет собой компактный и безопасный способ передачи данных между сторонами в виде JSON-объекта. Он широко используется для аутентификации и авторизации в веб-приложениях.

2.2.2 Структура JWT

JWT состоит из трёх частей, разделённых точками:

- **Header** (заголовок)
- **Payload** (полезная нагрузка)
- **Signature** (подпись)

Пример структуры JWT:

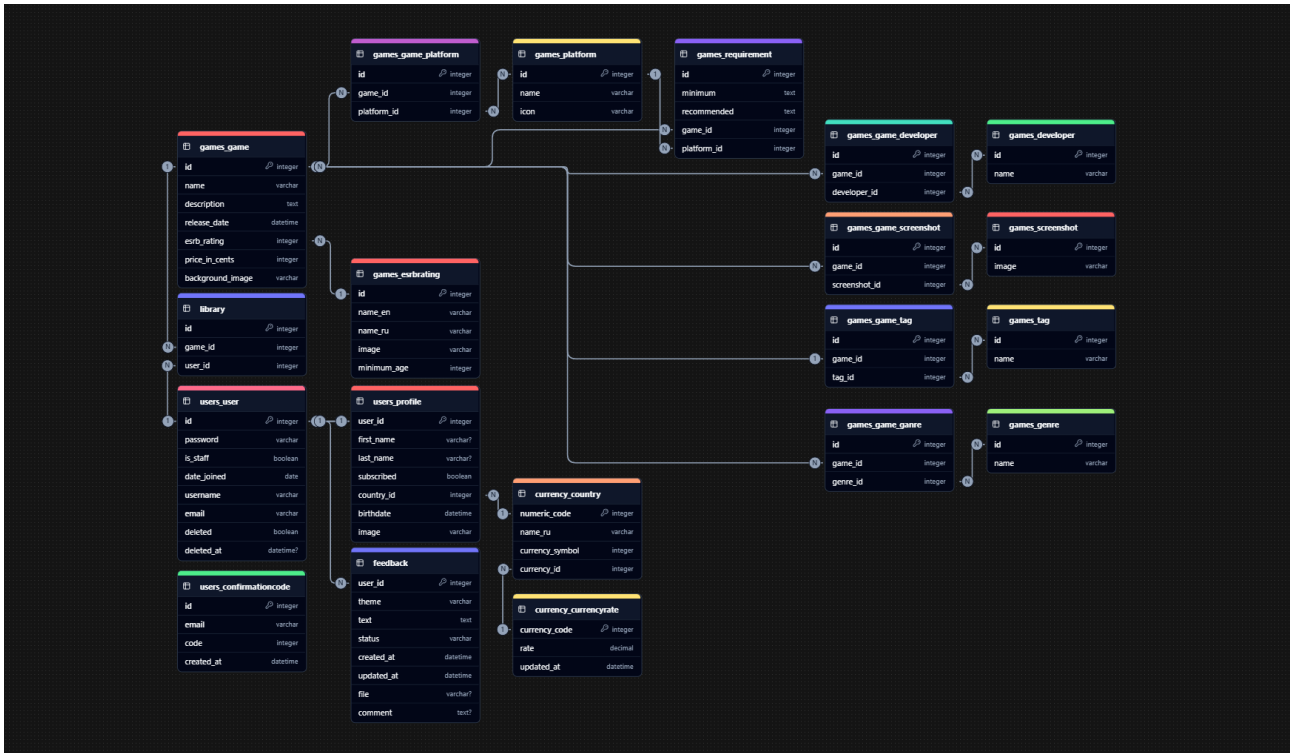
```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiIxMjMONTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwicm9sZXMiOiIiYWRtaW4iLCJ1c2VyIi119.SflKxwRJSMeKKF2QT4fwpMeJf36P0k6yJV_adQssw5c
```

Каждая часть кодируется в формате Base64 URL:

- **Header:** содержит информацию о типе токена (JWT) и алгоритме подписи (например, HS256).
- **Payload:** включает данные о пользователе и дополнительные метаданные (например, роли, ID пользователя, срок действия токена).
- **Signature:** создаётся с использованием алгоритма подписи и секретного ключа для проверки подлинности токена.

3 Backend

3.1 Структура базы данных



3.2 Подключенные инструменты

3.2.1 Simple JWT

Для аутентификации пользователей на Django мне необходим инструмент для работы с токенами. Одним из популярных инструментов для этого является пакет `django-rest-framework-simple-jwt`, который реализует аутентификацию с использованием JSON Web Token (JWT). Simple JWT предоставляет эндпоинты для получения и обновления токенов.

В данном проекте Simple JWT имеет следующие настройки:

```
1 SIMPLE_JWT = {
2     'ACCESS_TOKEN_LIFETIME': timedelta(minutes=5),
3     'REFRESH_TOKEN_LIFETIME': timedelta(days=50),
4     'ROTATE_REFRESH_TOKENS': True,
5     'BLACKLIST_AFTER_ROTATION': True,
6     'UPDATE_LAST_LOGIN': False,
7
8     'ALGORITHM': 'HS256',
9     'VERIFYING_KEY': None,
10    'AUDIENCE': None,
11    'ISSUER': None,
```



```

12     'JWK_URL': None,
13     'LEEWAY': 0,
14
15     'AUTH_HEADER_TYPES': ('Bearer',),
16     'AUTH_HEADER_NAME': 'HTTP_AUTHORIZATION',
17     'USER_ID_FIELD': 'id',
18     'USER_ID_CLAIM': 'user_id',
19     'USER_AUTHENTICATION_RULE':
20         'rest_framework_simplejwt.authentication.default_
user_authentication_rule',
21
22     'AUTH_TOKEN_CLASSES':
23         ('rest_framework_simplejwt.tokens.AccessToken',),
24     'TOKEN_TYPE_CLAIM': 'token_type',
25     'TOKEN_USER_CLASS':
26         'rest_framework_simplejwt.models.TokenUser',
27     'JTI_CLAIM': 'jti',
28
29     'SLIDING_TOKEN_REFRESH_EXP_CLAIM': 'refresh_exp',
30     'SLIDING_TOKEN_LIFETIME': timedelta(minutes=5),
31     'SLIDING_TOKEN_REFRESH_LIFETIME':
32         timedelta(days=1),
33 }

```

- **Жизненный цикл токенов:**

- **ACCESS_TOKEN_LIFETIME** – время жизни access-токена 5 минут;
- **REFRESH_TOKEN_LIFETIME** – время жизни refresh-токена 50 дней;
- **ROTATE_REFRESH_TOKENS** – при обновлении access-токена создаётся новый refresh-токен;
- **BLACKLIST_AFTER_ROTATION** – старые refresh-токены попадают в чёрный список после обновления, это сделано для большей безопасности;
- **SLIDING_TOKEN_LIFETIME** – время жизни sliding-токена 5 минут;
- **SLIDING_TOKEN_REFRESH_LIFETIME** – время жизни sliding refresh-токена 1 день.

- **Криптографические параметры:**

- **ALGORITHM** – алгоритм шифрования токена (HMAC с SHA-256). HMAC с SHA-256 – это алгоритм для создания криптографических подписей. Он берёт данные токена, добавляет к нему секретный ключ, применяет хэш-функцию SHA-256, в итоге на выходе получается HMAC (Hash-based Message Authentication Code) – цифровая подпись, защищающая данные от подделки;
 - **VERIFYING_KEY** – ключ для верификации токена;
 - **JWK_URL** – URL для загрузки JSON Web Key.
- **Аутентификация и авторизация:**
 - **AUTH_HEADER_TYPES** – тип заголовка авторизации;
 - **AUTH_HEADER_NAME** – имя заголовка с токеном;
 - **USER_ID_FIELD** – поле, идентифицирующее пользователя;
 - **USER_ID_CLAIM** – claim (ключ в payload токена), в котором передаётся user_id;
 - **USER_AUTHENTICATION_RULE** – правило аутентификации пользователей;
 - **TOKEN_TYPE_CLAIM** – claim, указывающий тип токена;
 - **TOKEN_USER_CLASS** – класс пользователя, связанного с токеном;
 - **JTI_CLAIM** – уникальный идентификатор токена (используется для blacklist).
 - **Дополнительные параметры:**
 - **UPDATE_LAST_LOGIN** – не обновлять поле last_login при использовании токена;
 - **LEEWAY** – дополнительное время для валидации токена (учитывает возможную разницу времени между серверами).

3.2.2 Simple Mail Transfer Protocol

SMTP (Simple Mail Transfer Protocol) — это протокол для отправки электронной почты. Данный протокол применяется в проекте для отправки электронных писем пользователям о коде подтверждения или для почтовой рассылки.

В качестве почтового хоста в проекте применяется smtp.gmail.com, также для данного проекта была создана специальная электронная почта gamestore.django.app от которого совершаются в отправки писем.

3.2.3 Celery

Celery — это асинхронный менеджер задач для Python. Он позволяет выполнять фоновые задачи (например, обработку данных, отправку email) без блокировки основного приложения. Для работы с Django применяется пакет `django-celery-beat`. Он создаёт новые таблицы в базе данных для создания с разными временными периодами выполнения.

Celery необходим брокер для хранения информации о задаче, в проекте в роли брокера используется Redis. Для работы celery необходимо запустить 2 модуля: `beat` и `worker`. Celery beat просматривает информацию в специальной таблице базы данных для поиска задач, которые необходимо выполнить, после нахождения задачи, он передаёт в Redis эту задачу. Celery worker в свою очередь выполняет все задачи, которые были переданы в Redis.

Всего в проекте 2 задачи. Первая – `send_subscription_emails`. Эта задача имеет периодичность выполнения раз в день. Она находит всех пользователей, которые имеют статус "подписан после чего отправляет всем найденным пользователям почтовую рассылку. Вторая – `delete_old_users`. Данная задача каждый день ищет пользователей со статусом "удалённый" и проверяет дату приобретения этого статуса, пользователь получил статус "удалённый" более 30 дней назад, то информация о пользователе стирается из базы данных.

3.2.4 Stripe API

Stripe API – это мощный API для работы с платежами, подписками и финансами в веб-приложениях. В проекте он применяется, как со стороны фронт-энда, так и бекэнда.

В клиентской части создаются специальные компоненты для ввода данных о банковской карте пользователя. После эти данные специальным образом конвертируются для безопасности сохранности данных.

Таким образом, в серверную часть уже передаются данные не самой карты, а некоторая информация, которую Stripe расшифровывает для последующей работы с ним. После совершается транзакция, а приобретённый продукт передаётся пользователю.

3.3 Роуты для обращения к API

Серверная часть сайта разделён на 6 приложений, каждая из которых выполняет свою задачу: `currency`, `feedback`, `games`, `library`, `payments`, `users`.

3.3.1 currency

GET: /currency/currency_rates/

Описание: Возвращает все объекты модели CurrencyRate.

Тело ответа:

```
1 [
2   {
3     "currency_code": "USD",
4     "rate": "1.000000",
5     "updated_at": "2024-12-02T12:55:54.013000Z"
6   },
7   ...
8 ]
```

GET: /currency/currency_rates/get/{currency_code}/

Описание: Возвращает определённый объект модели CurrencyRate в соответствии с currency_code.

Тело ответа:

```
1 {
2   "currency_code": "USD",
3   "rate": "1.000000",
4   "updated_at": "2024-12-02T12:55:54.013000Z"
5 }
```

GET: /currency/country/

Описание: Возвращает все объекты модели Country.

Тело ответа:

```
1 [
2   {
3     "numeric_code": 520,
4     "name_ru": "Напы",
5     "currency_symbol": "$",
6     "currency": "AUD"
7   },
8   ...
9 ]
```

GET: /currency/country/get/{numeric_code}/

Описание: Возвращает определённый объект модели Country в соответствии с numeric_code.

Тело ответа:

```
1 {  
2     "numeric_code": 520,  
3     "name_ru": "Найпы",  
4     "currency_symbol": "$",  
5     "currency": "AUD"  
6 }
```

3.3.2 feedback

GET: /feedback/feedback/

Описание: Возвращает все объекты модели Feedback, доступно только администраторам.

Тело ответа:

```
1 [  
2     {  
3         "id": 29,  
4         "theme": "Some theme",  
5         "text": "Some text",  
6         "file":  
7         "http://127.0.0.1:8000/static/files/file.  
8         file",  
9         "status": "Some status",  
10        "created_at":  
11        "2025-01-26T10:52:30.333019Z",  
12        "updated_at":  
13        "2025-01-26T16:04:07.857400Z",  
14        "user": 1,  
15        "comment": "Some comment"  
16     },  
17     ...  
18 ]
```

GET: /feedback/feedback/get/{user_id}/

Описание: Возвращает все объекты модели Feedback отправленные определённым пользователем в соответствии с указанным ID пользователя user_id.

Тело ответа:

```
1 [
2   {
3     "id": 29,
4     "theme": "Some theme",
5     "text": "Some text",
6     "file":
7       "http://127.0.0.1:8000/static/files/file.
8       file",
9     "status": "Some status",
10    "created_at":
11      "2025-01-26T10:52:30.333019Z",
12    "updated_at":
13      "2025-01-26T16:04:07.857400Z",
14    "user": 1,
15    "comment": "Some comment"
16  },
17  ...
18 ]
```

GET: /feedback/feedback/get_by_id/{feedback_id}/

Описание: Возвращает определённый объект модели Feedback в соответствии с feedback_id.

Тело ответа:

```
1 {
2     "id": 29,
3     "theme": "Some theme",
4     "text": "Some text",
5     "file":
6         "http://127.0.0.1:8000/static/files/file.file",
7     "status": "Some status",
8     "created_at": "2025-01-26T10:52:30.333019Z",
9     "updated_at": "2025-01-26T16:04:07.857400Z",
10    "user": 1,
11    "comment": "Some comment"
12 }
```

POST: /feedback/create/{user_id}/

Описание: Создаёт новый объект модели Feedback от пользователя в соответствии с user_id.

Тело запроса:

```
1 {
2     "theme": "Some theme",
3     "text": "Some text",
4     "file": "file.file"
5 }
```

Тело ответа в случае успеха:

```
1 {
2     "message": "Feedback created"
3 }
```

Тело ответа в случае провала:

```
1 {
2     "details": "Some error"
3 }
```

PUT: /feedback/feedback/update/{feedback_id}/

Описание: Обновляет данные объекта модели Feedback в соответствии с feedback_id.

Тело запроса:

```
1 {  
2     "theme": "Some theme",  
3     "text": "Some text",  
4     "file": "file.file"  
5 }
```

Тело ответа в случае успеха:

```
1 {  
2     "message": "Feedback updated"  
3 }
```

Тело ответа в случае провала:

```
1 {  
2     "details": "Some error"  
3 }
```

DELETE: /feedback/feedback/delete/{feedback_id}/

Описание: Удаляет данные объекта модели Feedback в соответствии с feedback_id.

Тело ответа в случае успеха:

```
1 {  
2     "message": "Feedback deleted"  
3 }
```

Тело ответа в случае провала из-за статуса, не равному "Отправлено":

```
1 {  
2     "message": "Feedback can't be deleted"  
3 }
```

Тело ответа в случае других причин провала:

```
1 {  
2     "details": "Some error"  
3 }
```


3.3.3 games

GET: /games/platform/

Описание: Возвращает все объекты модели Platform.

Тело ответа:

```
1 [
2   {
3     "id": 4,
4     "name": "Windows",
5     "icon":
6       "http://127.0.0.1:8000/static/icons/windows-
7       icon.png"
8   },
9   ...
10 ]
```

GET: /games/platform/get/{platform_id}/

Описание: Возвращает определённый объект модели Platform в соответствии с platform_id.

Тело ответа:

```
1 {
2   "id": 4,
3   "name": "Windows",
4   "icon":
5     "http://127.0.0.1:8000/static/icons/windows-i
6     con.png"
7 }
```

GET: /games/esrb_rating/

Описание: Возвращает все объекты модели ESRBRating.

Тело ответа:

```
1 [
2   {
3     "id": 1,
4     "name_en": "Everyone 10 and older",
5     "name_ru": "C 10 let",
6     "image":
7       "http://127.0.0.1:8000/static/icons/ESRB-
8       Everyone-10.png",
9     "minimum_age": 10
10  },
11  ...
12 ]
```

GET: /games/esrb_rating/get/{esrb_rating_id}/

Описание: Возвращает определённый объект модели ESRBRating в соответствии с esrb_rating_id.

Тело ответа:

```
1 {
2   "id": 1,
3   "name_en": "Everyone 10 and older",
4   "name_ru": "C 10 let",
5   "image":
6     "http://127.0.0.1:8000/static/icons/ESRB-Every
7     one-10.png",
8   "minimum_age": 10
9 }
```

GET: /games/genre/

Описание: Возвращает все объекты модели Genre.

Тело ответа:

```
1 [  
2   {  
3     "id": 1,  
4     "name": "Racing"  
5   },  
6   ...  
7 ]
```

GET: /games/genre/get/{genre_id}/

Описание: Возвращает определённый объект модели Genre в соответствии с genre_id.

Тело ответа:

```
1 {  
2   "id": 1,  
3   "name": "Racing"  
4 }
```

GET: /games/tag/

Описание: Возвращает все объекты модели Tag.

Тело ответа:

```
1 [  
2   {  
3     "id": 1,  
4     "name": "Survival"  
5   },  
6   ...  
7 ]
```

GET: /games/tag/get/{tag_id}/

Описание: Возвращает определённый объект модели Tag в соответствии с tag_id.

Тело ответа:

```
1 {  
2     "id": 1,  
3     "name": "Survival"  
4 }
```

GET: /games/developer/

Описание: Возвращает все объекты модели Developer.

Тело ответа:

```
1 [  
2     {  
3         "id": 10,  
4         "name": "Rockstar Games"  
5     },  
6     ...  
7 ]
```

GET: /games/developer/get/{developer_id}/

Описание: Возвращает определённый объект модели Developer в соответствии с developer_id.

Тело ответа:

```
1 {  
2     "id": 10,  
3     "name": "Rockstar Games"  
4 }
```

GET: /games/screenshot/

Описание: Возвращает все объекты модели Screenshot.

Тело ответа:

```
1 [  
2   {  
3     "id": 1827221,  
4     "image":  
5       "https://media.rawg.io/media/screenshots/  
6       a7c/a7c43871a54bed6573a6a429451564ef.jpg"  
7   },  
8   ...  
9 ]
```

GET: /games/screenshot/get/{screenshot_id}/

Описание: Возвращает определённый объект модели Screenshot в соответствии с screenshot_id.

Тело ответа:

```
1 {  
2   "id": 1827221,  
3   "image":  
4     "https://media.rawg.io/media/screenshots/a7c/  
5     a7c43871a54bed6573a6a429451564ef.jpg"  
6 }
```

GET: /games/game/?page={page_number}

Описание: Возвращает все объекты модели Game с пагинацией на странице page_number, на каждой странице 39 объектов.

Тело ответа:

```
1 {
2   "total_count": 9999,
3   "total_pages": 257,
4   "page": 2,
5   "next":
6     "http://127.0.0.1:8000/games/game/?page=3",
7   "previous":
8     "http://127.0.0.1:8000/games/game/",
9   "results": [
10    {
11      "id": 19710,
12      "name": "Half-Life 2: Episode One",
13      "background_image":
14        "https://media.rawg.io/media/games/7a2/7a
15        2500ee8b2c0e1ff268bb4479463dea.jpg",
16      "description": "<p>Description...</p>",
17      "esrb_rating": 6,
18      "release_date": "2006-06-01",
19      "price_in_cents": 0,
20      "platforms": [4, 5, 6],
21      "genres": [2, 4],
22      "tags": [8, 13, 30, 31, 32, 42, 62, 63,
23        110, 111, 118, 119, 172, 193, 232, 319,
24        11669, 40833, 40834, 40839, 40845,
25        40847, 40849],
26      "screenshots": [185831, 185832, 185833,
27        185834, 185835],
28      "developers": [1612, 23342]
29    },
30    ...
31  ]
32 }
```

GET: /games/games/get/{user_id}/?page={page_number}

Описание: Возвращает определённый объект модели Game, которые отсутствуют в библиотеке у пользователя с ID user_id, с пагинацией на странице page_number, на каждой странице 39 объектов.

Необязательный параметры:

- [&platforms={platform_id1}&platforms={platform_id2}[...]] – параметры для фильтрации игр по платформам, возвращает только те игры, где в качестве платформы указан хотя бы один из перечисленных platform_id1[, platform_id2[, ...]];
- [&genres={genre_id1}&genres={genre_id2}[...]] – параметры для фильтрации игр по жанрам, возвращает только те игры, где в качестве жанра указан хотя бы один из перечисленных genre_id1[, genre_id2[, ...]];
- [&tags={tag_id1}&tags={tag_id2}[...]] – параметры для фильтрации игр по тэгам, возвращает только те игры, где в качестве тэга указан хотя бы один из перечисленных tag_id1[, tag_id2[, ...]];
- [&developers={developer_id1}&developers={developer_id2}[...]] – параметры для фильтрации игр по разработчикам, возвращает только те игры, где в качестве разработчиков указан хотя бы один из перечисленных developer_id1[, developer_id2[, ...]];
- &name={game_title} – параметр для фильтрации игр по названию, возвращает только те игры, где название включает в себя game_title.

Тело ответа:

```
1 {
2   "total_count": 9999,
3   "total_pages": 257,
4   "page": 2,
5   "next":
6     "http://127.0.0.1:8000/games/game/?page=3",
7   "previous":
8     "http://127.0.0.1:8000/games/game/",
9   "results": [
10    ...
11  ]
12 }
```

GET: /games/games/get/{user_id}/?page={page_number}

Продолжение...

```
1 ...
2 {
3     "id": 19710,
4     "name": "Half-Life 2: Episode One",
5     "background_image":
6     "https://media.rawg.io/media/games/7a2/7a
7 2500ee8b2c0e1ff268bb4479463dea.jpg",
8     "description": "<p>Description...</p>",
9     "esrb_rating": 6,
10    "release_date": "2006-06-01",
11    "price_in_cents": 0,
12    "platforms": [4, 5, 6],
13    "genres": [2, 4],
14    "tags": [8, 13, 30, 31, 32, 42, 62, 63,
15            110, 111, 118, 119, 172, 193, 232, 319,
16            11669, 40833, 40834, 40839, 40845,
17            40847, 40849],
18    "screenshots": [185831, 185832, 185833,
19                    185834, 185835],
20    "developers": [1612, 23342]
21 },
22 ...
23 }
```


GET: /games/game/get/{game_id}

Описание: Возвращает определённый объект модели Game в соответствии с game_id.

Тело ответа:

```
1 {
2   "id": 19710,
3   "name": "Half-Life 2: Episode One",
4   "background_image":
5     "https://media.rawg.io/media/games/7a2/7a2500
6     ee8b2c0e1ff268bb4479463dea.jpg",
7   "description": "<p>Description...</p>",
8   "esrb_rating": 6,
9   "release_date": "2006-06-01",
10  "price_in_cents": 0,
11  "platforms": [4, 5, 6],
12  "genres": [2, 4],
13  "tags": [8, 13, 30, 31, 32, 42, 62, 63, 110,
14          111, 118, 119, 172, 193, 232, 319, 11669,
15          40833, 40834, 40839, 40845, 40847, 40849],
16  "screenshots": [185831, 185832, 185833,
17                  185834, 185835],
18  "developers": [1612, 23342]
```

GET: /games/random_games/{user_id}

Описание: Возвращает 6 случайных объекта модели Game, которые отсутствуют в библиотеке пользователя с ID user_id.

Тело ответа:

```
1 [
2   {
3     "id": 19710,
4     "name": "Half-Life 2: Episode One",
5     "background_image":
6       "https://media.rawg.io/media/games/7a2/7a
7 2500ee8b2c0e1ff268bb4479463dea.jpg",
8     "description": "<p>Description...</p>",
9     "esrb_rating": 6,
10    "release_date": "2006-06-01",
11    "price_in_cents": 0,
12    "platforms": [4, 5, 6],
13    "genres": [2, 4],
14    "tags": [8, 13, 30, 31, 32, 42, 62, 63,
15             110, 111, 118, 119, 172, 193, 232, 319,
16             11669, 40833, 40834, 40839, 40845,
17             40847, 40849],
18    "screenshots": [185831, 185832, 185833,
19                    185834, 185835],
20    "developers": [1612, 23342]
21  },
22  ...
23 ]
```

GET: /games/check_game_in_library/{user_id}/{game_id}

Описание: Возвращает true или false в зависимости от того, есть ли у пользователя с ID user_id в библиотеке игра с ID game_id.

Тело ответа в случае, если есть:

```
1 {  
2   "in_library": true  
3 }
```

Тело ответа в случае, если нет:

```
1 {  
2   "in_library": false  
3 }
```

GET: /games/requirement/

Описание: Возвращает все объекты модели Requirement.

Тело ответа:

```
1 [  
2   {  
3     "id": 1,  
4     "minimum": "Minimum:OS: Windows 10 64 Bit,  
5               Windows 8.1 64 Bit, Windows 8 64...",  
6     "recommended": "Recommended:OS: Windows 10  
7                     64 Bit, Windows 8.1 64 Bit, Windows...",  
8     "platform": 4,  
9     "game": 3498  
10  },  
11  ...  
12 ]
```

GET: /games/requirement/get/{game_id}/

Описание: Возвращает все объекты модели Requirement игры с ID game_id.

Тело ответа:

```
1 [
2   {
3     "id": 1,
4     "minimum": "Minimum:OS: Windows 10 64 Bit,
5               Windows 8.1 64 Bit, Windows 8 64...",
6     "recommended": "Recommended:OS: Windows 10
7                   64 Bit, Windows 8.1 64 Bit, Windows...",
8     "platform": 4,
9     "game": 3498
10  },
11  ...
12 ]
```

3.3.4 library

GET: /library/library/

Описание: Возвращает все объекты модели Library.

Тело ответа:

```
1 [
2   {
3     "user": 1,
4     "game": 50677
5   },
6   ...
7 ]
```

GET: /library/library/get/{user_id}/

Описание: Возвращает все объекты модели Library пользователя с ID user_id.

Тело ответа:

```
1 [  
2   {  
3     "user": 1,  
4     "game": 50677  
5   },  
6   ...  
7 ]
```

POST: /library/library/add/{user_id}/{game_id}/

Описание: Создаёт новый объект модели Library с игрой ID game_id для пользователя с ID user_id.

Тело ответа в случае успеха:

```
1 {  
2   "message": "Game added to library"  
3 }
```

Тело ответа в случае провала из-за наличия игры у пользователя:

```
1 {  
2   "error": "This game is already in the library"  
3 }
```

3.3.5 payments

POST: /payments/save_stripe_info/

Описание: Совершает запрос в приложение stripe для совершения транзакции.

Тело запроса:

```
1 {  
2     "email": "some_email@mail.com",  
3     "payment_method_id": 999,  
4     "price": 999  
5 }
```

Тело ответа в случае, если пользователь с указанной электронной почтой впервые совершает транзакцию:

```
1 {  
2     "message": "Success",  
3     "data": {  
4         "customer_id": 999,  
5         "extra_msg": ""  
6     }  
7 }
```

Тело ответа в случае, если пользователь с указанной электронной почтой уже совершал транзакцию:

```
1 {  
2     "message": "Success",  
3     "data": {  
4         "customer_id": 999,  
5         "extra_msg": "Customer already existed."  
6     }  
7 }
```

3.3.6 users

POST: /users/token/

Описание: Возвращает JSON Web Token с информацией о зарегистрированном пользователе.

Тело запроса:

```
1 {  
2   "email": "examole@mail.com"  
3   "password": "example_password"  
4 }
```

Тело ответа в случае успеха:

```
1 {  
2   "refresh":  
3     "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",  
4   "access":  
5     "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."  
6 }
```

Тело ответа в случае провала из-за неправильной почты или неправильного пароля:

```
1 {  
2   "detail": "No active account found with the  
3             given credentials"  
4 }
```

POST: /users/token/refresh/

Описание: Обновляет JSON Web Token пользователя, это необходимо из-за ограниченного срока действия токена.

Тело запроса:

```
1 {  
2   "refresh":  
3     "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."  
}
```

Тело ответа в случае успеха:

```
1 {  
2   "refresh":  
3     "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",  
4   "access":  
5     "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."  
}
```

Тело ответа в случае провала из-за некорректного refresh токена:

```
1 {  
2   "detail": "Token is invalid or expired",  
3   "code": "token_not_valid"  
4 }
```

Тело ответа в случае провала из-за заблокированного refresh токена:

```
1 {  
2   "detail": "Token is blacklisted",  
3   "code": "token_not_valid"  
4 }
```


POST: /users/register/

Описание: Создаёт новые объекты User и соответствующий Profile.

Тело запроса:

```
1 {
2   "username": "user",
3   "email": "exaple@mail.com",
4   "password": "example_password",
5   "password2": "example_password",
6   "first_name": "Name",
7   "last_name": "Surname",
8   "birthdate": 30.12.2000,
9   "country": 643
10 }
```

Тело ответа в случае успеха:

```
1 {
2   "username": "user",
3   "email": "exaple@mail.com"
4 }
```

Тело ответа в случае провала из-за занятых электронной почты и/или имени пользователя:

```
1 {
2   "username": [
3     "user with this username already exists."
4   ],
5   "email": [
6     "user with this email already exists."
7   ]
8 }
```

Тело ответа в случае провала из-за пустого значения одного из полей:

```
1 {
2   "username": [
3     "This field may not be blank."
4   ],
5   ...
6 }
```

POST: /users/check_password/{user_id}/

Описание: Проверяет правильность пароля, соответствующий пользователю с ID user_id.

Тело запроса:

```
1 {  
2   "password": "example_password"  
3 }
```

Тело ответа в случае успеха:

```
1 {  
2   "details": "Password is valid"  
3 }
```

Тело ответа в случае провала из-за невалидного пароля:

```
1 {  
2   "details": "Password is invalid"  
3 }
```

Тело ответа в случае провала из-за отсутствия пользователя с ID user_id:

```
1 {  
2   "details": "User not found"  
3 }
```

POST: /users/check_email/

Описание: Проверяет, что переданная электронная почта не занята другим пользователем.

Тело запроса:

```
1 {  
2   "email": "example@mail.com"  
3 }
```

Тело ответа в случае успеха:

```
1 {  
2   "details": "Email does not exist"  
3 }
```

Тело ответа в случае провала из-за существования пользователя с данной электронной почтой:

```
1 {  
2   "details": "Email already exists"  
3 }
```

Тело ответа в случае провала из-за существования пользователя с данной электронной почтой, имеющий статус "удалённый":

```
1 {  
2   "details": "User is deleted"  
3 }
```

POST: /users/check_username/

Описание: Проверяет, что переданное имя пользователя не занята другим пользователем.

Тело запроса:

```
1 {  
2   "username": "username"  
3 }
```

Тело ответа в случае успеха:

```
1 {  
2   "details": "Username does not exist"  
3 }
```

Тело ответа в случае провала из-за существования пользователя с данным именем:

```
1 {  
2   "details": "Username already exists"  
3 }
```

Тело ответа в случае провала из-за существования пользователя с данным именем, имеющий статус "удалённый":

```
1 {  
2   "details": "User is deleted"  
3 }
```

PUT: /users/update/{user_id}/

Описание: Обновляет данные пользователя, кроме электронной почты и пароля.

Тело запроса:

```
1 {
2   "username": "username",
3   "password": "password",
4   "first_name": "Name",
5   "last_name": "Surname",
6   "birthdate": 30.12.2000,
7   "country": 643,
8   "image": "file.file"
9 }
```

Тело ответа в случае успеха:

```
1 {
2   "details": "Data updated"
3 }
```

Тело ответа в случае провала из-за неверного пароля:

```
1 {
2   "details": "Invalid password"
3 }
```

Тело ответа в случае провала из-за существования иного пользователя с указанным именем:

```
1 {
2   "details": "User with this username already
3             exists"
```

PUT: /users/subscribe/{user_id}/

Описание: Изменяет статус подписки пользователя с ID user_id на противоположный.

Тело ответа в случае подписки:

```
1 {  
2   "details": "User subscribed true"  
3 }
```

Тело ответа в случае отписки:

```
1 {  
2   "details": "User subscribed false"  
3 }
```

DELETE: /users/delete/{user_id}/

Описание: Делает статус пользователя с ID user_id удалённым.

Тело ответа в случае успеха:

```
1 {  
2   "detail": "User deleted"  
3 }
```

Тело ответа в случае провала из-за отсутствия пользователя с указанным user_id:

```
1 {  
2   "detail": "User not found."  
3 }
```

Тело ответа в случае провала из-за запроса от иного пользователя:

```
1 {  
2   "detail": "You can't delete stranger account."  
3 }
```

PUT: /users/recover/

Описание: Убирает статус "удалённый" у пользователя.

Тело запроса:

```
1 {  
2   "recover": "email or username"  
3 }
```

Тело ответа в случае успеха:

```
1 {  
2   "User recovered"  
3 }
```

Тело ответа в случае провала из-за отсутствия пользователя с таким данным:

```
1 {  
2   "details": "User not found"  
3 }
```

Тело ответа в случае провала из-за иных причин:

```
1 {  
2   "details": "error: Some error"  
3 }
```

POST: /users/create_confirmation_code/

Описание: Создаёт или обновляет объект модели `ConfirmationCode`, соответствующий указанной электронной почте, и отправляет электронное письмо на эту почту с кодом подтверждения.

Тело запроса:

```
1 {  
2   "email": "example@mail.com"  
3 }
```

Тело ответа в случае успеха:

```
1 {  
2   "details": "Reset password code sent"  
3 }
```

Тело ответа в случае провала:

```
1 {  
2   "details": "error: Some error"  
3 }
```


POST: /users/check_confirmation_code/

Описание: Проверяет код подтверждения, соответствующий электронной почте.

Тело запроса:

```
1 {  
2   "email": "example@mail.com",  
3   "code": "1234"  
4 }
```

Тело ответа в случае успеха:

```
1 {  
2   "details": "Confirmation code is valid"  
3 }
```

Тело ответа в случае провала из-за невалидного кода:

```
1 {  
2   "details": "Confirmation code is invalid"  
3 }
```

Тело ответа в случае провала из-за отсутствия кода подтверждения для указанной электронной почты:

```
1 {  
2   "details": "Confirmation code not found"  
3 }
```

Тело ответа в случае провала из-за иных причин:

```
1 {  
2   "details": "error: Some error"  
3 }
```

PUT: /users/reset_password/

Описание: Обновляет пароль пользователя.

Тело запроса:

```
1 {  
2   "email": "exaple@mail.com",  
3   "password": "new_password"  
4 }
```

Тело ответа в случае усаеха:

```
1 {  
2   "details": "Password changed"  
3 }
```

Тело ответа в случае провала:

```
1 {  
2   "details": "error: Some error"  
3 }
```

PUT: /users/change_email/{user_id}/

Описание: Обновляет электронную почту пользователя в соответствии с user_id.

Тело запроса:

```
1 {  
2   "email": "example@mail.com"  
3 }
```

Тело ответа в случае успеха:

```
1 {  
2   "details": "Email changed"  
3 }
```

тело ответа в случае провала:

```
1 {  
2   "details": f"error: Some error"  
3 }
```